

用例建模与用户故事

软件工程与计算 II · 模块二 · 单元 3

从"系统该做什么"到"用户如何与系统对话"

课程	软件工程与计算 II
专业	软件工程
工具	SEECODER-SE 平台 · AI 辅助用例建模

今日学习路径

① 预习测试	② 用例概念	③ 用例图建模	④ 用例描述	⑤ AI生成与审查
回顾上节课内容 系统顺序图	什么用例？ 场景与用例关系	怎么画用例图？ 参与者·关系·边界	怎么写用例描述？ 10字段模板	LLM能自动生成吗？ 质量如何审查？

三个核心问题，今天结束时你应该能回答：

1. 用例和场景有什么区别？用例图怎么建？
2. 一份完整的用例描述应该包含哪些要素？
3. LLM 能自动生成用例吗？质量如何？

预习测试

根据 CD 商店场景，完成以下任务：

1. 建立系统顺序图：顾客查询 → 选定 CD → 查看详情 → 购买 → 离开
2. 写出一个用例的详细描述

预习测试 —— 系统顺序图



PART 1 · 用例概念

需求与用例 [SWEBOK 4.2]

传统方式的困境

- 需求是客户和开发者之间的**合同文档**——但难以捕获所有需求
- 自然语言编写的需求规格说明书，往往存在**二义性、不完整、不一致**等问题

Use Case 方法的诞生

- **1992 年**，Ivar Jacobson 在 Objectory 方法中提出 Use Case（用例）
- 从传统开发方法演化而来，天然适配**面向对象分析与设计（OOAD）**
- 核心理念：从**用户的角度**描述系统行为，而非从系统内部出发

用例的定义 [SWEBOK 4.2]

定义一 [Jacobson, 1992]

在系统和外部对象的交互当中所执行的**行为序列的描述**，包括各种不同的序列和错误的序列，它们能够联合提供一种**有价值的服务**。

定义二 [Cockburn, 2001]

用例描述了在不同条件下系统对某一用户的请求的**响应**。每一个行为序列被称为一个**场景 (Scenario)**。一个用例是多个场景的集合。

💡 **关键区分：** 场景 = 单条路径；用例 = 所有场景的集合

目标、交互与行为序列 [SWEBOK 4.2]

Essential Use Case vs Real Use Case

类型	Actor 动作	System 响应
Essential (本质用例)	Actor 表明身份	系统呈现可用选项
Real (真实用例)	顾客插入银行卡	系统提示输入 PIN 码
	顾客输入 PIN	系统验证并显示菜单

- **Essential Use Case**: 抽象描述, 不涉及具体技术实现
- **Real Use Case**: 具体描述, 包含界面与操作细节
- 需求阶段通常先写 Essential, 设计阶段再转为 Real

案例引入 —— 连锁超市管理系统

场景描述：收银员为了完成一次销售任务，会使用软件系统处理销售过程

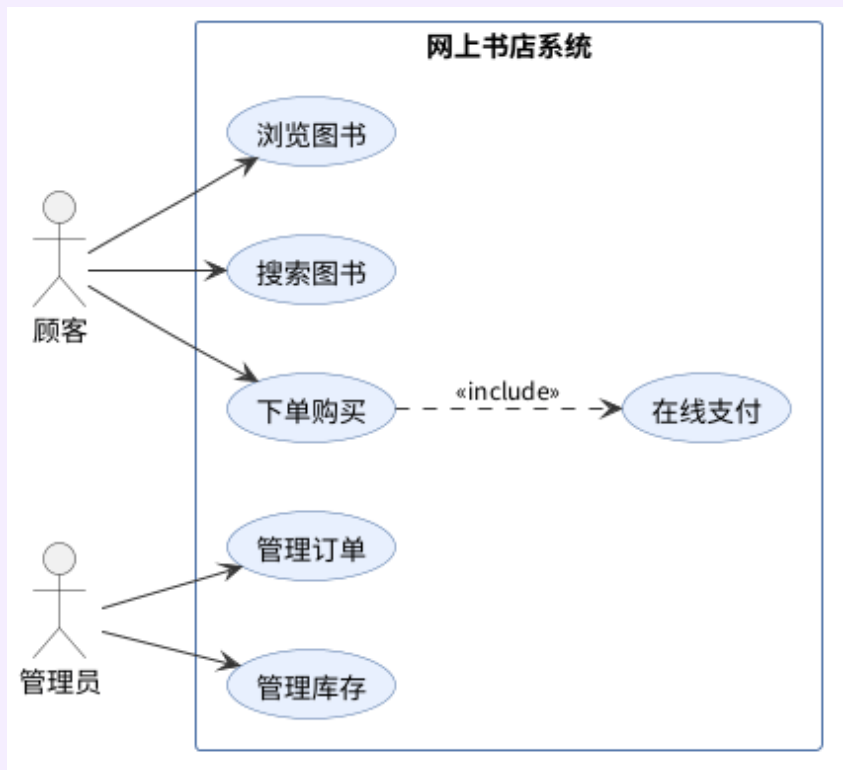
用例"销售处理"包含的场景

场景编号	场景描述
S1	正常流程：扫码 → 计价 → 收款 → 打印收据
S2	购买多个同样商品：输入商品标识和数量
S3	商品无法识别：系统提示错误
S4	商品退回：收银员在系统中处理退货
S5	顾客取消交易：收银员取消本次销售

 上述每一个行为都是一个**场景**，所有场景联合构成用例"销售处理"

PART 2 · 用例图建模

用例图基本元素 [SWEBOK 4.4]



四要素说明

元素	符号	说明
用例	椭圆	系统提供的一项有价值的服务
参与者	小人图标	与系统交互的外部角色
关系	连线	参与者与用例之间的关系
系统边界	矩形框	划定系统内外的边界

Actor（参与者） [SWEBOK 4.4]

An actor is a **role** that a user or other system plays with respect to the system.

参与者的关键特征

- 一个 Actor 可以代表**多个用户**（如"收银员"代表所有收银员）
- 一个用户可以扮演**多个角色**（如某人既是收银员又是管理员）
- Actor **不一定是人**——外部系统、硬件设备、定时器都可以是 Actor

如何识别参与者？

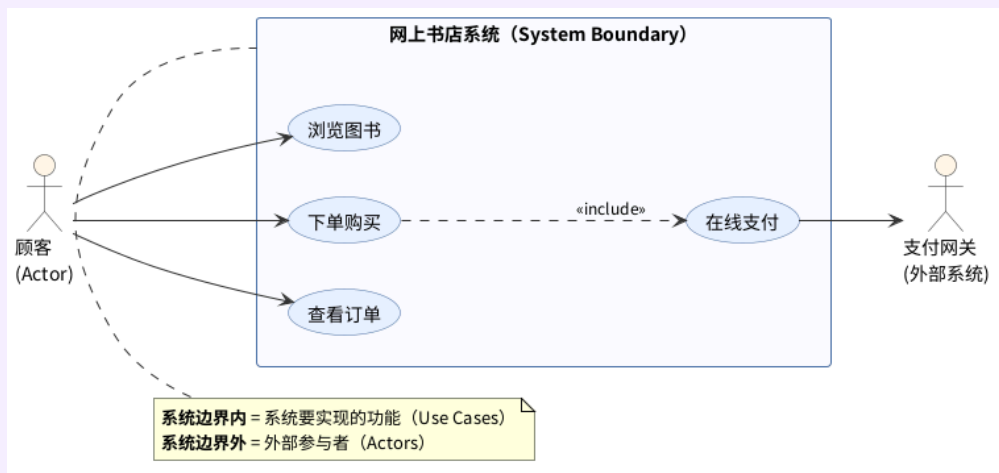
问题	示例
谁使用系统的主要功能？	收银员、顾客
谁需要系统支持日常工作？	总经理、客户经理
谁维护和管理系统？	系统管理员
系统需要与哪些外部系统交互？	支付网关、库存系统

识别参与者的常见误区 [SWEBOK 4.4]

误区	错误示例	正确做法
将系统内部组件当参与者	"数据库"、"缓存服务器"	数据库是系统内部组成部分，不是外部实体
用具体人名代替角色	"张三"、"李经理"	应写"收银员"、"总经理"——参与者是 角色 不是个人
遗漏非人类参与者	只列出人类用户	外部系统（支付网关）、定时器（每日对账）同样是参与者
参与者粒度不当	"青年顾客"和"老年顾客"分两个	除非功能需求不同，否则统一为"顾客"

System Boundary（系统边界） [SWEBOK 4.4]

什么在系统内部？什么在系统外部？



核心规则

- **Actors** 始终在系统边界**外部**
- **Use Cases** 始终在系统边界**内部**
- 系统边界帮助明确**项目范围 (Scope)**

边界划定的意义

- 明确哪些功能由**系统实现**
- 明确哪些角色是**外部交互方**
- 避免范围蔓延 (Scope Creep)

一个系统可以有多张用例图，每张聚焦不同的子系统边界

Relationship (关系) [SWEBOK 4.4]

关系类型	符号	说明
Association (关联)	实线	参与者与用例之间的交互
Include (包含)	虚线 + <<include>>	一个用例 必须 包含另一个用例的行为
Extend (扩展)	虚线 + <<extend>>	一个用例在 特定条件 下扩展另一个用例
Generalization (泛化)	实线 + 空心三角	参与者之间的继承关系

(基础用例) <-----<<include>>----- (被包含用例)

必须执行被包含的行为

(基础用例) <-----<<extend>>----- (扩展用例)

在特定条件下才执行扩展行为

总经理 <----- 分店经理

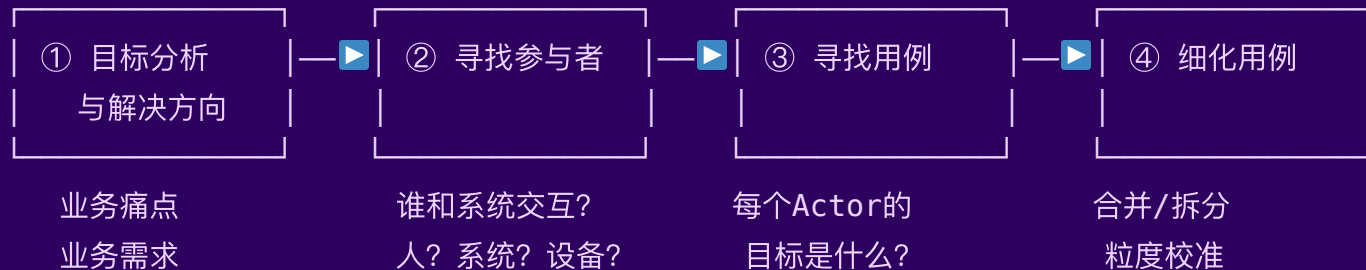
(泛化: 分店经理继承总经理的权限)

Include vs Extend 辨析 [SWEBOK 4.4]

对比维度	Include（包含）	Extend（扩展）
是否必须执行	必须执行	有条件执行
方向	基础用例 → 被包含用例	扩展用例 → 基础用例
典型场景	多个用例共享同一段行为	基础用例在特定条件下的额外行为
举例	销售处理 <<include>> 打印收据	销售处理 <<extend>> VIP 折扣计算

💡 记忆口诀：Include = 必须做；Extend = 可能做

用例图的建立过程 [SWEBOK 4.4]



步骤	关键问题	产出
① 目标分析	业务痛点是什么? 期望达到什么效果?	业务需求列表
② 寻找参与者	谁和系统交互?	Actor 列表
③ 寻找用例	每个 Actor 的工作目标是什么?	初版用例列表
④ 细化用例	粒度是否合适? 是否需要合并/拆分?	最终用例图

案例 —— 连锁超市目标分析

背景：xxx 连锁商店是一家刚发展起来的小型连锁商店

问题编号	业务痛点
问题 1	手工作业销售迟缓，顾客排队等待时间长
问题 2	商品品种增多，库存失控（积压、缺货、报废）
问题 3	市场竞争加大，需要降低运营成本

 目标分析的核心：从业务痛点出发，明确系统要解决的问题

业务需求

编号	业务需求	对应痛点
BR1	商品积压、缺货和报废减少 50%	问题 2
BR2	销售人员工作效率提高 50%	问题 1
BR3	运营成本降低 15%	问题 3
BR4	销售额度提高 20%	问题 3

💡 业务需求必须是可量化、可验证的，不能只写"提高效率"

系统功能（特性）—— 库存与营销

编号	系统功能	支撑的业务需求
SF1	分析商品库存	BR1
SF2	调整销售商品	BR1, BR4
SF3	制定促销手段	BR4
SF4	联合促销	BR4
SF5	销售竞争分析	BR3, BR4

系统功能（特性）—— 人员与销售

编号	系统功能	支撑的业务需求
SF6	员工管理	BR2
SF7	商品入库出库	BR1
SF8	发展会员	BR4
SF9	积分兑换赠送	BR4
SF10	销售与退货	BR2

寻找参与者与用例

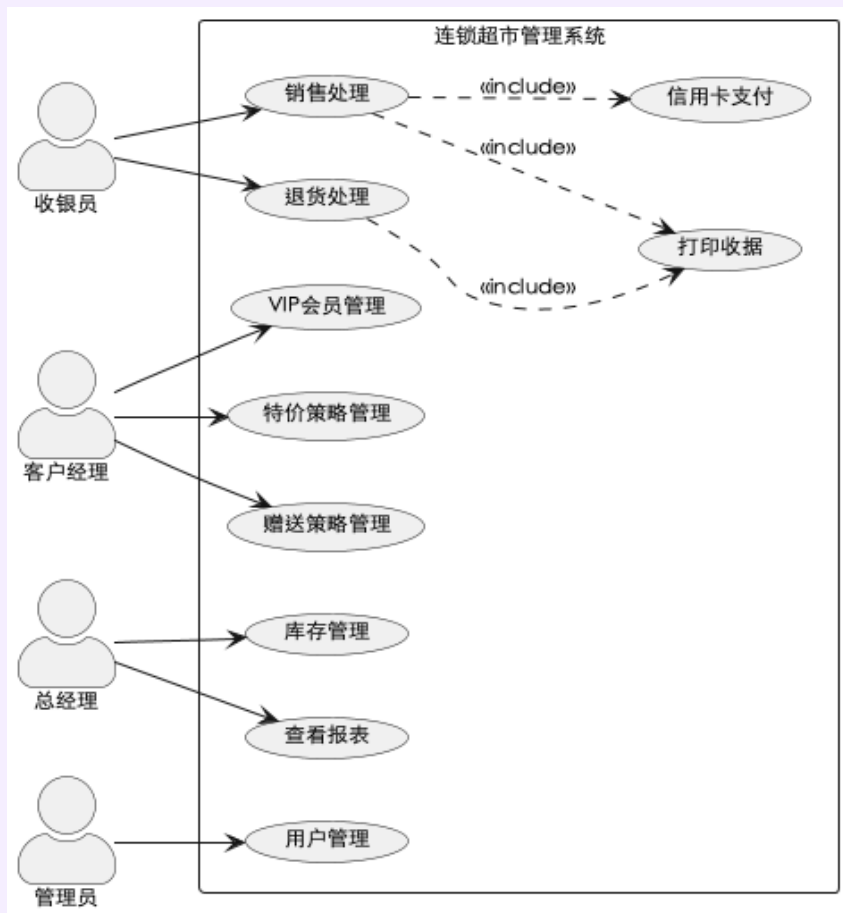
每个用户的任务（目标）都是一个独立用例

参与者	目标（用例）
总经理	产品调整、特价策略制定、赠送策略制定、库存分析
客户经理	会员管理、库存管理
收银员	销售处理、退货
管理员	用户管理

识别用例的关键问题

- 每个参与者需要系统提供什么**服务**？
- 系统需要对哪些**外部事件**做出响应？
- 系统的日常**维护操作**有哪些？

超市销售系统用例图（初版）



识别出的参与者

- 收银员、总经理、客户经理、管理员

识别出的用例

- 销售处理、退货处理
- 库存管理、会员管理
- 特价策略制定、赠送策略制定
- 查看销售报表

这是初版——下一步需要检查粒度并细化

细化用例

判断标准

用例应对一个业务事件，由一个用户发起，在一个连续时间段内完成

操作	原用例	调整后	原因
合并	特价策略制定 + 赠送策略制定	→ 销售策略制定	业务目的相同，属于同一业务事件
拆分	会员管理	→ 发展会员 + 礼品赠送	两个不同业务事件，不同时间触发
拆分	库存管理	→ 商品出库 + 商品入库 + 库存分析	三个独立业务事件

常见错误

✗ 错误 1：将用例细化为单个操作

- 不要将"会员管理"拆成"增加会员""修改会员""删除会员"3 个用例
- 这些只是同一个用例内部的**操作步骤**，不是独立的业务事件

✗ 错误 2：将同一业务目标拆成不同用例

- "特价策略制定"和"赠送策略制定"服务于同一个业务目标 → 应合并

✗ 错误 3：将无业务价值的内容作为用例

- "登录" → 不是独立的业务目标，只是前置操作
- "数据验证" → 系统内部行为，用户不直接感知
- "连接数据库" → 技术实现细节，不是用户目标

💡 判断标准：用例必须为参与者提供**可感知的、有价值的结果**

用例建模常见错误（补充） [Cockburn 2001]

#	错误类型	错误示例	正确做法
④	混入技术实现细节	"系统执行 SELECT * FROM books"	写"系统查询图书信息"
⑤	遗漏替代场景	只写正常流程	每一步都要考虑"可能出什么错? "
⑥	写入界面设计细节	"点击左上角蓝色搜索按钮"	写"顾客提交搜索请求"

用例描述应独立于界面设计和技术实现，使用业务专家能理解的语言

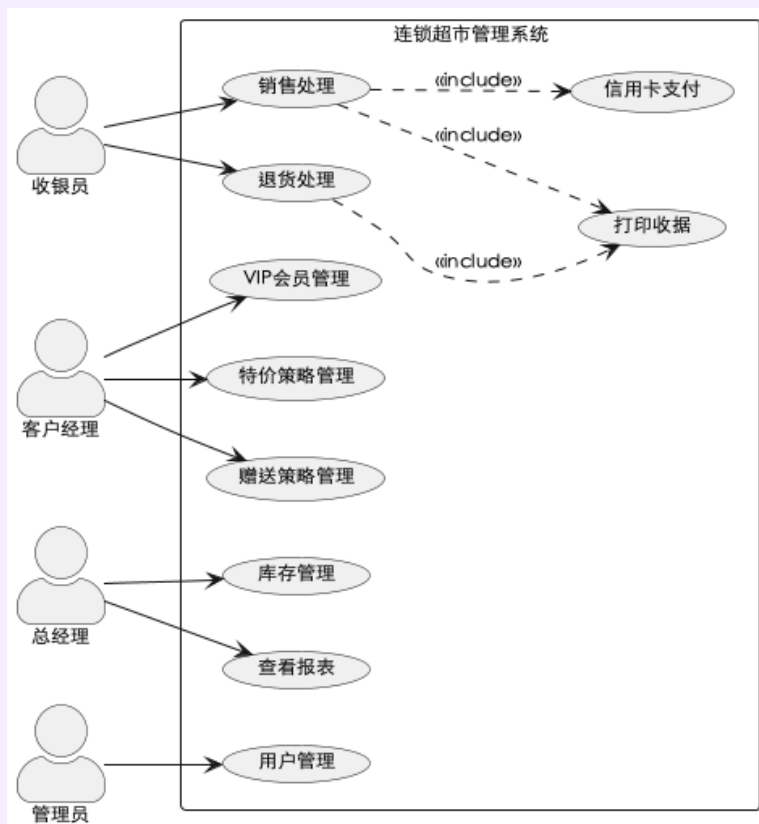
用例的粒度判断 —— Cockburn 三层次 [Cockburn 2001]

层次	别名	说明	示例
概要层	☁ 云层	涉及多个用户目标的长期过程	"管理书店运营"——粒度过粗
用户目标层	🌊 海平面	参与者的一个完整业务目标	"购买图书""搜索图书"—— 最佳粒度
子功能层	🐟 水下	实现目标的一个步骤	"输入密码""验证信用卡"——粒度过细

判断经验法则

- 该用例能否在**一次会话**中完成？
- 完成后用户是否有一种****"任务达成"**的满足感**？
- 如果两个都是"是" → 用户目标层 

细化后的完整用例图

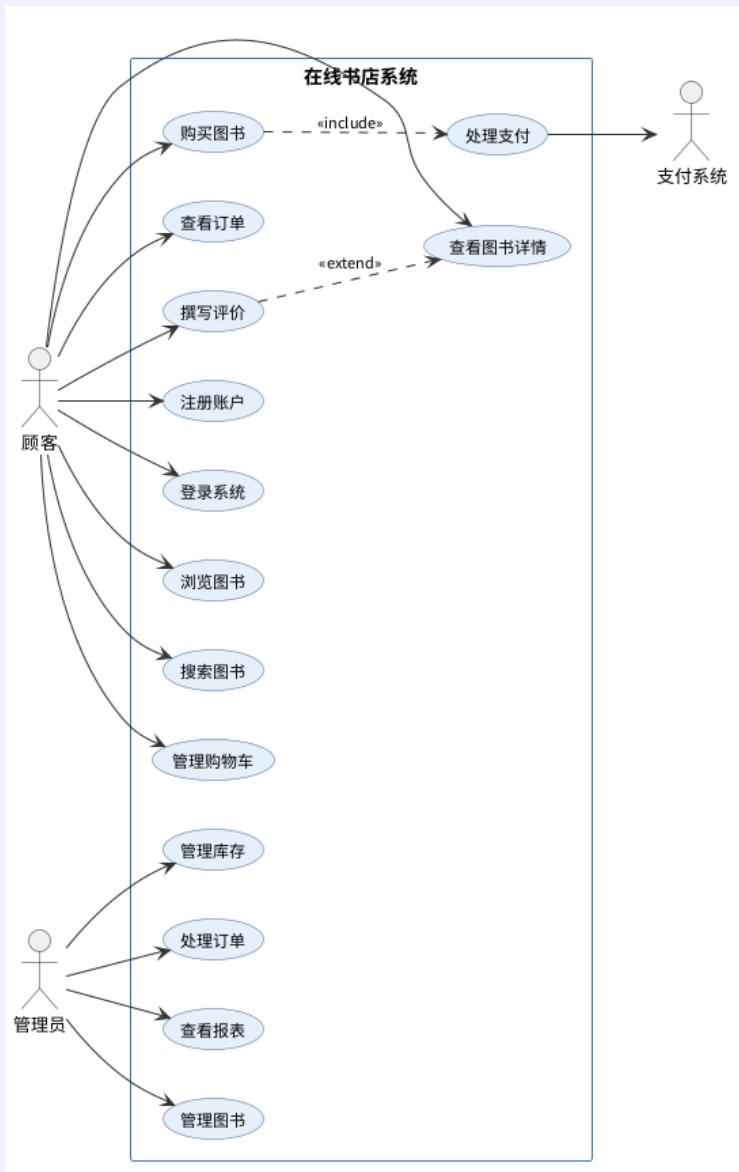


对比初版的变化

- 合并：特价策略 + 赠送策略 → 销售策略制定
- 拆分：会员管理 → 发展会员 + 礼品赠送
- 拆分：库存管理 → 出库 + 入库 + 库存分析

每个用例对应一个业务事件

案例对照 —— 在线书店系统用例图



13 个用例 + 3 个参与者

- 顾客 (9 个用例)
- 管理员 (4 个用例)
- 支付系统 (被 include)

关系

- 购买图书 <<include>> 处理支付
- 撰写评价 <<extend>> 查看图书详情

在线书店用例图 —— 解读

参与者	用例
顾客	注册账户、登录系统、浏览图书、搜索图书、查看图书详情、管理购物车、购买图书、查看订单、撰写评价
管理员	管理图书、管理库存、处理订单、查看报表
支付系统	处理支付（被"购买图书" <code><<include>></code> ）

关系解读

- 购买图书 `--<<include>>-->` 处理支付：购买**必然**包含支付
- 撰写评价 `--<<extend>>-->` 查看图书详情：查看详情时**可选**撰写评价

PART 3 · 用例描述

用例描述模板（10 个字段）—— 基本信息 [SWEBOK 4.2]

字段	内容描述
ID	用例的唯一标识（如 UC1、UC2）
名称	对用例内容的精确描述
参与者	描述系统的参与者和每个参与者的目标
触发条件	标识启动用例的事件
前置条件	用例能够正常启动和工作的系统状态条件
后置条件	用例执行完成后的系统状态条件

用例描述模板（10 个字段）—— 行为与补充 [SWEBOK 4.2]

字段	内容描述
正常流程	在常见和符合预期的条件下，系统与外界的行为交互序列
扩展流程	用例中可能发生的其他场景（异常、分支）
特殊需求	和用例相关的其他特殊需求，尤其是非功能性需求
补充说明	其他需要记录的信息

前 6 个字段描述上下文，后 4 个字段描述行为与约束

UC1 销售处理 —— 基本信息 [SWEBOK 4.2]

字段	内容
ID	UC1
名称	销售处理
参与者	收银员——目标是快速、正确地完成商品销售
触发条件	顾客携带商品到达销售点
前置条件	收银员必须已经被识别和授权
后置条件	存储销售记录，更新库存，打印收据

 **前置条件**描述的是系统在用例开始前必须满足的状态，
不是用例自身要执行的步骤（如"登录"不应出现在正常流程中）

UC1 销售处理 —— 正常流程 (1/2) [SWEBOK 4.2]

步骤	行为描述
1	收银员开始一次新的销售
2	如果是 VIP 顾客，收银员输入客户编号
3	收银员输入商品标识
4	系统记录商品并显示商品和赠品信息
5	系统显示已购入的商品清单和赠品清单
6	收银员结束输入，系统计算总价

UC1 销售处理 —— 正常流程 (2/2) [SWEBOK 4.2]

步骤	行为描述
7	系统根据赠送策略补充赠品清单
8	收银员请顾客支付
9	顾客支付，收银员输入现金数额
10	系统给出应找的余额
11	系统记录销售信息并更新库存
12	系统打印收据

UC1 销售处理 —— 扩展流程 [SWEBOK 4.2]

编号	扩展条件	处理方式
2a	非法客户编号	系统提示错误并拒绝输入
3a	非法商品标识	系统提示错误并拒绝输入
3b	多个相同类别商品	收银员手工输入商品标识和数量
3-7a	顾客要求删除商品	收银员输入标识删除，系统更新列表和总价
3-7b	顾客要求取消交易	收银员在系统中取消交易
6a	没有可执行的特价策略	系统计算并生成抵价券

 **扩展流程的编号规则：** 数字表示从正常流程的哪一步分支出去。
如 "3-7a" 表示在步骤 3 到 7 之间的任意时刻都可能发生。

正常流程 vs 扩展流程 [SWEBOK 4.2]

正常流程 (Main Success Scenario)

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11 → 12

|

▼

2a: 非法ID

3b: 多件

|

▼

3a: 非法商品

3-7a: 删除商品

3-7b: 取消交易

|

▼

6a: 无特价策略

- 每一条扩展都是一个独立的**场景 (Scenario)**
- 所有场景的集合 = 用例"销售处理"

💡 写扩展流程时, 要考虑**每一步可能出错的情况**——这是人工审查的重点

案例 —— 在线书店"购买图书"用例（1/3）

字段	内容
用例名称	购买图书（Purchase Books） UC-07
参与者	顾客（主）、支付系统（辅）
前置条件	顾客已登录；购物车中至少有一件商品
后置条件（成功）	订单已创建，库存已扣减，支付已完成
后置条件（失败）	订单未创建，购物车不变，库存未扣减

主成功场景（节选）

步骤	行为描述
1	顾客选择"去结算"
2	系统显示购物车商品清单（名称、单价、数量、小计）及总金额
3	系统检查所有商品库存，确认充足
4	顾客确认商品清单
5-7	顾客填写/选择收货地址，系统验证

案例 —— 在线书店"购买图书"用例（2/3）

主成功场景（续）

步骤	行为描述
8-9	顾客选择支付方式
10	系统生成订单，记录商品、地址、支付方式
11	系统向支付系统发送支付请求
12-13	支付系统处理支付并返回成功结果
14-15	系统更新订单为"已支付"，扣减库存
16-17	系统显示确认页，发送通知

替代场景（节选）

编号	触发条件	处理
3a	部分商品库存不足	提示顾客，可修改/删除/取消
12a	支付失败	通知顾客，可重试或取消
12b	支付超时	订单标记"待支付"，可稍后继续
*a	任意步骤顾客取消	确认后放弃结算，购物车不变

案例 —— 在线书店"购买图书"用例（3/3）

业务规则

编号	规则
BR1	同一商品最大购买数量不超过 99 件
BR2	订单生成后 30 分钟内未支付，系统自动取消并释放库存
BR3	订单总金额满 99 元免运费，否则收取运费 8 元

 注意 `*a` 编号——表示在任意步骤都可能发生的替代场景，这是 Cockburn 模板的通配写法

案例 —— 在线书店"管理购物车"用例

字段	内容
用例名称	管理购物车（Manage Shopping Cart） UC-06
参与者	顾客
前置条件	顾客已登录
后置条件	购物车内容已更新

主成功场景

步骤	行为描述
1	顾客在图书详情页选择"加入购物车"
2	系统将图书添加到购物车，数量默认为 1
3	系统显示购物车已更新提示

替代场景

编号	触发条件	处理
2a	该图书已在购物车中	数量 +1，不重复添加
*a	顾客修改某商品数量	验证数量合法性，更新小计和总额
*b	顾客移除某商品	删除商品，重新计算总额

验收标准驱动的需求规格 [SWEBOK 4.3]

为什么需要验收标准？

- 自然语言需求天生模糊，但测试用例语言是精确的
- 思路：用"测试怎么写"来倒逼需求精确化

ATDD (Acceptance Test–Driven Development)

1. 选择一个用户故事
2. 开发者 + 业务专家 + 测试 共同编写验收测试用例
3. 先写测试，再写代码，直到所有验收用例通过

BDD —— Given-When-Then 格式 [SWEBOK 4.3]

用户故事：As a 顾客，I want to 退回商品，So that 获得退款

场景 1：正常退货

Given 顾客在 30 天内购买了商品
And 商品未拆封
When 顾客提交退货申请
Then 系统生成退货单
And 退款金额等于购买金额

场景 2：超期退货

Given 顾客在 30 天前购买了商品
When 顾客提交退货申请
Then 系统拒绝退货
And 显示"已超过退货期限"

BDD 的价值与实践 [SWEBOK 4.3]

Given-When-Then 解决了什么问题?

传统需求写法	BDD 写法	优势
"系统应支持退货"	Given/When/Then 场景集	消除歧义
模糊、开放式	精确、可执行	可直接转为自动化测试
验收标准不明确	场景即验收标准	开发/测试/业务三方对齐

BDD 与用例描述的关系 [SWEBOK 4.3]

用例描述（正常流程 + 扩展流程）



等价转换



BDD 场景集（Given-When-Then）



自动化框架（Cucumber / Behave / SpecFlow）



可执行的验收测试

💡 BDD 不是替代用例描述，而是用例描述的**精确化补充**。

在 SDD（Spec 驱动开发）中，Acceptance Criteria 通常采用 BDD 格式。

PART 4 · AI 辅助用例生成

LLM 自动生成用户故事 —— 前沿工具

GeneUS (2024)

- 首个基于 GPT-4 从需求文档**自动创建用户故事**的工具
- 使用 **Quality User Story (QUS)** 框架评估生成质量

QUS 质量维度	含义
原子性 (Atomic)	一个用户故事只描述一件事
独立性 (Independent)	故事之间不相互依赖
可协商性 (Negotiable)	不包含具体实现细节
有价值 (Valuable)	对用户有明确价值
可估算 (Estimable)	可以估算工作量
小巧 (Small)	一个迭代内可完成

研究发现：**RaT prompting** 可将用户故事的重复率降至 **0.5%** (arXiv, 2024)

LLM 生成用例 vs 人工 —— 对比实验

课堂练习

1. 给出电梯系统业务描述：

一栋 20 层的写字楼需要安装智能电梯调度系统，支持乘客呼梯、楼层选择、开关门、超载报警等功能。

2. 用 LLM 生成用例描述（正常流程 + 扩展流程）

3. 人工审查要点：

审查维度	关键问题
扩展流程	是否完整覆盖了边界情况？
前后置条件	是否精确且可验证？
设计细节	是否混入了实现层面的描述？
用例粒度	是否合适（不过粗也不过细）？

审查 AI 生成用例的检查清单

检查项	典型 AI 问题	审查方法
用例粒度	过细（拆成 CRUD）或过粗（多个业务事件合一）	检查是否对应单一业务事件
参与者识别	遗漏非人类参与者（外部系统、定时器）	对照系统上下文图
扩展流程	仅覆盖 "happy path"，边界情况不足	逐步检查每一步可能的异常
前后置条件	过于笼统或遗漏关键状态	验证条件是否可测试
业务价值	包含无业务价值的伪用例（如"登录"）	追问"对用户价值是什么？"
关系使用	include/extend 关系使用混乱	检查是否满足"必须/可能"准则

 **核心原则：**AI 擅长生成正常流程，但扩展流程和边界条件需要人工补充

课堂总结

今日核心知识点

主题	要点
用例概念	用例是场景的集合，描述系统与参与者的交互
用例图建模	目标分析 → 找参与者 → 找用例 → 细化
用例描述	10 字段模板，重点是正常流程和扩展流程
ATDD/BDD	Given-When-Then 精确化需求，可直接转为自动化验收测试
AI 辅助	AI 可快速生成初稿，但扩展流程和边界条件需人工补充

回答三个核心问题

1. **用例 vs 场景**：场景是一条路径，用例是所有路径的集合；用例图通过 Actor + Use Case + 关系建模
2. **完整用例描述**：ID、名称、参与者、触发条件、前/后置条件、正常流程、扩展流程、特殊需求
3. **LLM 生成用例**：正常流程质量尚可，扩展流程覆盖不足，需人工审查和补充

课后作业

以本组项目为背景，完成以下任务：

任务	要求
任务 1	建立系统用例图（标注所有参与者、用例、关系）
任务 2	选择 2 个核心用例，写出完整用例描述（10 字段）
任务 3	用 LLM 生成同样 2 个用例的描述，对比人工版本，记录差异
任务 4	提交对比分析报告（差异表 + 200 字反思）

提交要求

- 用例图可使用 PlantUML / Mermaid / Draw.io 等工具绘制
- 用例描述使用课堂模板（10 字段）
- 对比报告需包含：AI 生成版本、人工版本、差异分析表

下节课预告

需求分析方法与 AI 辅助建模

主题	内容
概念类图	从用例描述中提取领域概念和关系
顺序图	对象之间的交互时序建模
状态图	对象生命周期状态转换
AI 辅助	LLM + PlantUML / Mermaid 自动生成 UML 图
评测体系	MermaidSeqBench 评测 LLM 生成顺序图的质量

预习任务：了解 PlantUML 或 Mermaid 的基本语法，尝试用 LLM 生成一张简单的类图

本节课结束

"用例不是功能列表，而是用户与系统对话的剧本。"